Global Air Pollution Indicator

Aim :To build a Univariate Linear Regression model using the Global Air Pollution Dataset to predict AQI Value from a single independent variable and evaluate the performance of the model using regression metrics.

- Independent variable (X): PM2.5 AQI Value
- Dependent variable (y)**:** AQI Value

Step 1:

Before starting the regression analysis, we import the Python libraries required for data handling and numerical operations.

- pandas is used to load, inspect, and manipulate the dataset.
- numpy is used for numerical and mathematical calculations.
- matplotlib.pyplot is used for plotting and will be useful later if we decide to visualize the regression results.

Step 2:

After importing the required libraries, the next step is to load the dataset into Python. Since the Global Air Pollution Dataset is stored in CSV (Comma-Separated Values) format, Pandas' read_csv() function is used to read the file.

The data is stored in a Pandas DataFrame, which is a tabular data structure consisting of rows and columns. A DataFrame makes it easier to inspect, manipulate, and prepare the dataset for machine learning.

We also use head() to display the first few rows and verify that the dataset has been loaded correctly.

Step 3: Understanding the Dataset

Before building the regression model, we need to understand the structure, size, data types, and statistical characteristics of the dataset. This helps us identify which variables are suitable for regression and whether there are any data-quality issues.

Function explanations:

- df.shape → Returns the number of rows and columns in the dataset.
- df.dtypes → Displays the data type of each column.
- df.describe() → Provides statistical summaries of numerical columns.
- df.isnull().sum() → Counts the missing values in each column.
- df.duplicated().sum() → Counts the number of duplicate rows.

From your output, we already know:

- Dataset size: 23463 × 12
- Numerical columns: 5
- Missing Country values: 427
- Missing City values: 1
- Missing values in our numerical columns: 0
- Duplicate rows: 0

So our selected regression variables are ready to use.

Step 4: Selecting the Variables

Theory:

Univariate linear regression uses one independent variable to predict one dependent variable. Therefore, we need to select one input feature (X) and one target variable (y).

For our analysis:

- X = PM2.5 AQI Value
- y = AQI Value

We are choosing PM2.5 AQI Value as the predictor because it is a numerical variable and represents the air-quality impact of particulate matter, while AQI Value will be the value we want to predict.

- print() → Displays the specified values in the output.
- X.shape → Shows the dimensions of the input feature.
- y.shape → Shows the dimensions of the target variable.

Output:

(23463, 1)

(23463,)

The 1 confirms that we are using exactly one independent variable, making this a univariate regression problem.

One independent variable, PM2.5 AQI Value, was selected as the predictor (X), while AQI Value was selected as the dependent variable (y). Since only one independent variable is used, the analysis follows the univariate linear regression approach.

Step 5: Splitting the Dataset into Training and Testing Sets

Before training the regression model, the dataset is divided into two parts: a training set and a testing set. The training set is used to teach the model the relationship between X and y, while the testing set is used to evaluate how well the trained model performs on unseen data.

We will use 80% of the data for training and 20% for testing.

Function explanations:

- train_test_split() → Randomly divides the features and target into training and testing sets.
- test_size=0.2 → Allocates 20% of the data to testing and 80% to training.
- random_state=42 → Ensures the same split is obtained every time the code is executed.

The dataset was divided into training and testing subsets using an 80:20 ratio. The training data is used to train the regression model, while the testing data is reserved for evaluating its performance on unseen observations. A fixed random state was used to ensure reproducibility.

Step 6: Creating the Linear Regression Model

After splitting the dataset, we create a Linear Regression model. The model will learn a straight-line relationship between the independent variable, PM2.5 AQI Value, and the dependent variable, AQI Value.

The general equation is:

where:

- $\hat{y}$ → predicted AQI Value
- b_0 → intercept
- b_1 → coefficient (slope)
- x → PM2.5 AQI Value

At this step, we only create the model. The model has not learned the relationship yet. That happens in the next step when we use .fit().

Function/code explanations:

- LinearRegression() → Creates a linear regression model.
- model = → Stores the created model in the variable model.

Step 7: Training the Linear Regression Model

Now we train the model using the training data. During training, the model learns the relationship between PM2.5 AQI Value and AQI Value by finding the best-fitting straight line through the training observations.

The model estimates the coefficient and intercept that minimize the difference between the actual AQI values and the values predicted by the regression line.

Function explanation:

- model.fit() → Trains the linear regression model using the training features and their corresponding target values

Step 8: Finding the Coefficient and Intercept

After training, the model has learned the parameters of the regression line:

The coefficient (b_1) represents how much the predicted AQI Value changes when PM2.5 AQI Value increases by 1 unit.

The intercept (b_0) represents the predicted AQI Value when PM2.5 AQI Value is 0.

Function/code explanations:

- model.coef_ → Returns the coefficient (slope) learned by the model.
- model.intercept_ → Returns the intercept learned by the model.
- [0] → Extracts the coefficient value from the one-element array.

Note: Although only one independent variable is used in univariate regression, X is kept in 2D format because Scikit-learn expects input features in the form (number of samples, number of features).

Step 9: Making Predictions

After training the model, we use it to predict the AQI Value for the observations in the test set. The model uses the learned coefficient and intercept to calculate a predicted value for each PM2.5 AQI Value.

Function explanation:

- model.predict() → Uses the trained model to generate predicted target values from the test features.

Step 10: Comparing Actual and Predicted Values

Comparing actual and predicted values allows us to see how closely the model's predictions match the real AQI values. This gives us an initial understanding of the model's prediction performance before calculating formal evaluation metrics.

Function explanations:

- pd.DataFrame() → Creates a new DataFrame from the actual and predicted values.
- y_test.values → Extracts the actual AQI values from the test set as an array.
- comparison.head(10) → Displays the first 10 actual and predicted values together.

Steo 11: Evaluating the Model

After making predictions, we need to measure how well the predicted AQI values match the actual AQI values.

For regression, we use error and goodness-of-fit metrics rather than classification accuracy.

We will calculate four commonly used metrics:

- MAE → Mean Absolute Error
- MSE → Mean Squared Error
- RMSE → Root Mean Squared Error
- R^2 → R-squared score
- mean_absolute_error() → Calculates the average absolute difference between actual and predicted values.
- mean_squared_error() → Calculates the average squared difference between actual and predicted values.
- np.sqrt() → Calculates the square root of a value.
- r2_score() → Measures how much of the variation in the target is explained by the model.

For interpretation:

- MAE → lower is better
- MSE → lower is better
- RMSE → lower is better
- R^2 → closer to 1 is better

Step 11: Constructing the Regression Equation

After training the model, the learned intercept and coefficient can be substituted into the standard linear regression equation:

The following code displays the equation using the values learned by the model.

Function/code explanations:

- f"..." → Creates a formatted string that allows Python values to be inserted into the text.
- model.intercept_ → Provides the intercept of the regression equation.
- model.coef_[0] → Provides the coefficient of the PM2.5 AQI Value feature.
- :.2f → Rounds the numerical value to 2 decimal places.

Step 12: Residual Analysis

A residual is the difference between the actual value and the value predicted by the model. Residuals help us understand the prediction errors made by the regression model.

A good regression model generally has residuals that are relatively small and do not show a clear systematic pattern.

residuals = y_test - y_pred

print(residuals.head())

Function/code explanation:

- y_test - y_pred → Calculates the residual for each test observation by subtracting the predicted value from the actual value.

Step 13: Final Interpretation and Conclusion

The final step is to interpret the model's results in the context of the problem. Instead of only reporting numerical metrics, we explain what those values tell us about the relationship between PM2.5 AQI Value and AQI Value.

For our model, the R^2 score of 0.9694 indicates a strong linear relationship between the selected predictor and target on the test data. The MAE of 5.62 indicates that the model's predictions differ from the actual AQI values by about 5.62 units on average.

MAE measures the average magnitude of prediction errors, while RMSE gives greater weight to larger errors; therefore, both metrics are useful for assessing the prediction error of the regression model.

Step 14: A scatter plot